hashlock.

Security Audit

LOCKON (DeFi)

September 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

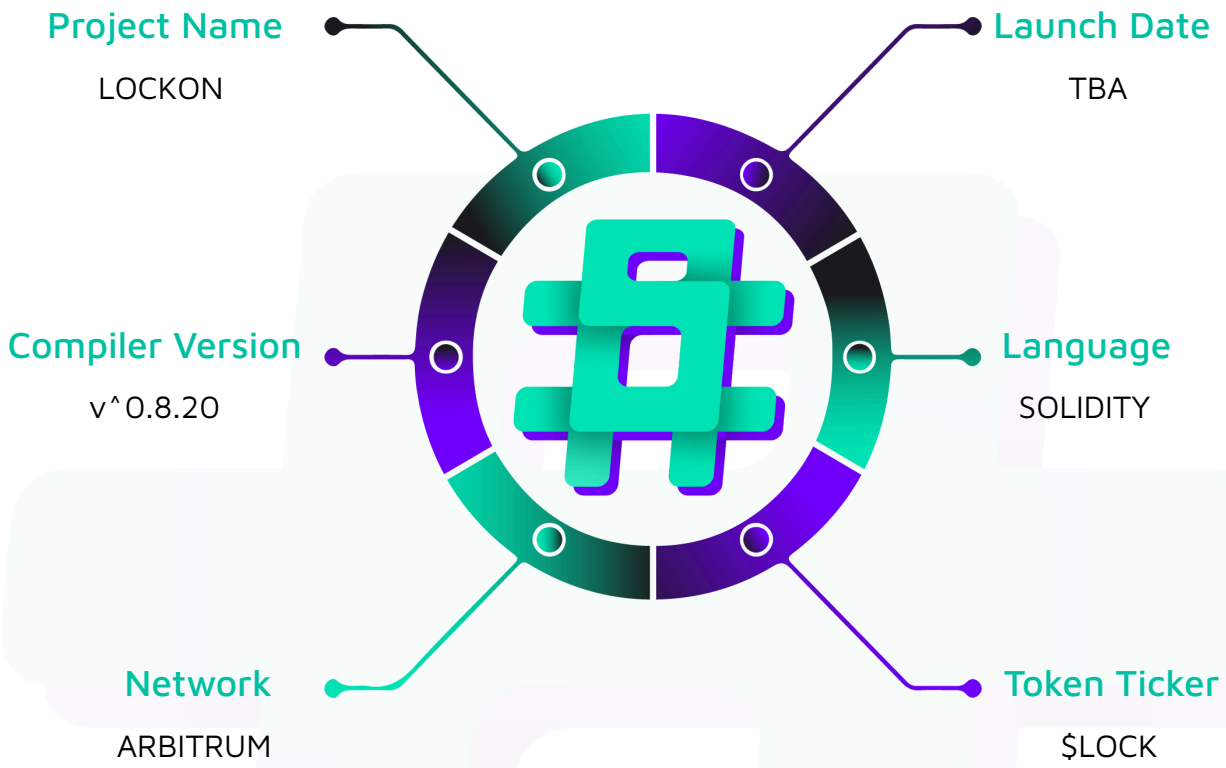
The LOCKON team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

LOCKON (lockon.finance) is a DeFi platform that builds dynamically managed investment INDEXes by analyzing on-chain behavior—identifying wallet addresses with strong profit histories and constructing portfolios from them. It offers choices like "Passive" and "Balance" INDEXes, tailored to users' risk preferences, and continuously adapts to market shifts. The project, founded in 2021 and based in Dubai, raised around $810K in seed funding in October 2023 to bolster its development and operations Watch. Notably, a thorough smart-contract security audit by Blaize rated its implementation highly secure (9.6/10), with only a medium-risk issue swiftly addressed.

Project Name: LOCKON
Project Type: Defi
Compiler Version: ^0.8.20
Website: www.lockon.finance
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the LOCKON project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**LOCKON Smart Contracts**
Platform	**Arbitrum / Solidity**
Audit Date	**September, 2025**
Contract 1	Airdrop.sol
Contract 2	IndexStaking.sol
Contract 3	LockonVesting.sol
Contract 4	LockStaking.sol
Contract 5	LockToken.sol
Audited GitHub Commit Hash	815bb9bb33c7a7f18309360ccb4f1ace2a39617c
Fix Review GitHub Commit Hash	2559660307ccd5c5b900b73acc9990c1a9be4942

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

2 Medium severity vulnerabilities

6 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Airdrop.sol - Is used to Airdrop Lock tokens to registered addresses, and after distribution, only registered addresses can claim those tokens	**Contract achieves this functionality.**
IndexStaking.sol - Provides different pools for user to stake different tokens to claim rewards - Reward distribution done by backend API	**Contract achieves this functionality.**
LockonVesting.sol - Core contract among all of them, because this is where actual transfer of reward/airdrop tokens to user happens	**Contract achieves this functionality.**
LockStaking.sol - Users can stake Lock tokens to receive rewards. The longer the duration of staking, the higher the rewards accumulated.	**Contract achieves this functionality.**
LockToken.sol - Core ERC20 Token with blacklist feature, that involved in every contract	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the LOCKON project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the LOCKON project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] IndexStaking#initialize - First Depositor Receives More Rewards Than Actual If Pool startTimestamp is in Future

Description

If the pool `startTimestamp` is in the future, `i.e.`, greater than the contract deployment time, then the first depositor in the pool accumulates more rewards, and it also increases `lockReward`, which means there will be more rewards assigned to the pool than actually intended.

Vulnerability Details

When a user deposits the pool gets updated, which means it calls `IndexStaking::updatePool`, which calls `getRewardMultipier` to calculate the amount of rewards to unlock.

```
  function updatePool(address _stakeToken) public {

            require(tokenPoolInfo[_stakeToken].stakeToken != IERC20(address(0)), "Index Staking: Pool do not exist");

        PoolInfo storage poolInfo = tokenPoolInfo[_stakeToken];

        if (poolInfo.totalStakedAmount == 0) {

            poolInfo.lastRewardTimestamp = block.timestamp;

            poolInfo.lastGeneratedReward = 0;

            return;

        }

        // Calculate the reward multiplier based on the difference in block timestamps

@>                      uint256 lockReward = getRewardMultiplier(_stakeToken, poolInfo.lastRewardTimestamp, block.timestamp);
```

```
        if (lockReward == 0) {
            return;
        }
        // Update state data
        currentRewardAmount = currentRewardAmount - lockReward;
        poolInfo.lastGeneratedReward = lockReward;
        poolInfo.lastRewardTimestamp = block.timestamp;
        emit PoolDataUpdated(
            msg.sender,
            _stakeToken,
            currentRewardAmount,
            poolInfo.lastGeneratedReward,
            poolInfo.lastRewardTimestamp
        );
    }
    function getRewardMultiplier(address _stakeToken, uint256 _from, uint256 _to) public
view returns (uint256) {
@>      uint256 multiplier = rewardTokenPerSecond(_stakeToken);
        uint256 rewardDistributed = (_to - _from) * multiplier;
        if (rewardDistributed < currentRewardAmount) {
            return rewardDistributed;
        }
        return currentRewardAmount;
    }
    function rewardTokenPerSecond(address _stakeToken) public view returns (uint256) {
        PoolInfo storage poolInfo = tokenPoolInfo[_stakeToken];
```

```
@>      return (currentRewardAmount * poolInfo.bonusRatePerSecond) / currentNumOfPools /
PRECISION;

    }
```

As we can see in the above calls, while calculating the `lockReward` variable, the `updatePool` calls `getRewardMultiplier`, which calculates the number of rewards to unlock based on `from` and `to` timestamps. But the issue here is that if `from = poolInfo.lastRewardTimestamp`, then the number of rewards that are unlocked is far greater than the intended rewards release.

That is because in the function `IndexStaking::initialize`, we can see that `lastRewardTimestamp` was set to `block.timestamp`. So if pool `startTimestamp` is greater than the `block.timestamp`, which is the most likely scenario, increases the likelihood to unlock more tokens, which essentially have sudden decrease in `currentRewardAmount` at first deposit of the pools.

```
     function initialize(

        address _owner,

        address _validator,

        address _lockonVesting,

        address _lockToken,

        uint256 _currentRewardAmount,

        string memory _domainName,

        string memory _signatureVersion,

        InitPoolInfo[] calldata pools

    ) external initializer {

        require(_owner != address(0), "Index Staking: owner is the zero address");

             require(_validator != address(0), "Index Staking: validator is the zero
address");

         require(_lockonVesting != address(0), "Index Staking: lockonVesting is the zero
address");
```

```
            require(_lockToken != address(0), "Index Staking: lockToken is the zero address");

        // Initialize the contract and set the owner

        // This function should be called only once during deployment

        __UUPSUpgradeable_init();

        __Ownable_init(_owner);

        __Pausable_init();

        EIP712Upgradeable.__EIP712_init(_domainName, _signatureVersion);

        __ReentrancyGuard_init();

        validatorAddress = _validator;

        lockonVesting = _lockonVesting;

        currentRewardAmount = _currentRewardAmount;

        lockToken = IERC20(_lockToken);

        uint256 len = pools.length;

        for (uint256 i; i < len;) {

            IERC20 stakeToken = pools[i].stakeToken;

             require(address(stakeToken) != address(0), "Index Staking: Zero address not allowed");

            require(

                  pools[i].bonusRatePerSecond != 0, "Index Staking: Pool bonus rate per second must be greater than 0"

            );

             require(pools[i].vestingCategoryId != 0, "Index Staking: Vesting category id must be greater than 0");

            tokenPoolInfo[address(stakeToken)] =

@>              PoolInfo(stakeToken, 0, 0, pools[i].bonusRatePerSecond, block.timestamp, pools[i].startTimestamp);
```

```
                                    stakeTokenToVestingCategoryId[address(stakeToken)]   =
pools[i].vestingCategoryId;

            unchecked {

                ++i;

            }

        }

        currentNumOfPools += len;

    }
```

Impact

Because of setting `lastRewardTimestamp` to `block.timestamp`, making the first user of the pool to receive more rewards than actually intended if `startTimestamp` is in the past, `i.e.`, less than `block.timestamp` while deploying.

Recommendation

Make the `lastRewardTimestamp` be set to the `startTimestamp` of the pool:

```
 function initialize(

        address _owner,

        address _validator,

        address _lockonVesting,

        address _lockToken,

        uint256 _currentRewardAmount,

        string memory _domainName,

        string memory _signatureVersion,

        InitPoolInfo[] calldata pools

    ) external initializer {

        require(_owner != address(0), "Index Staking: owner is the zero address");
```

```
            require(_validator != address(0), "Index Staking: validator is the zero address");
        require(_lockonVesting != address(0), "Index Staking: lockonVesting is the zero address");
            require(_lockToken != address(0), "Index Staking: lockToken is the zero address");
        // Initialize the contract and set the owner
        // This function should be called only once during deployment
        __UUPSUpgradeable_init();
        __Ownable_init(_owner);
        __Pausable_init();
        EIP712Upgradeable.__EIP712_init(_domainName, _signatureVersion);
        __ReentrancyGuard_init();
        validatorAddress = _validator;
        lockonVesting = _lockonVesting;
        currentRewardAmount = _currentRewardAmount;
        lockToken = IERC20(_lockToken);
        uint256 len = pools.length;
        for (uint256 i; i < len;) {
            IERC20 stakeToken = pools[i].stakeToken;
            require(address(stakeToken) != address(0), "Index Staking: Zero address not allowed");
            require(
                  pools[i].bonusRatePerSecond != 0, "Index Staking: Pool bonus rate per second must be greater than 0"
            );
            require(pools[i].vestingCategoryId != 0, "Index Staking: Vesting category id must be greater than 0");
```

```
            tokenPoolInfo[address(stakeToken)] =

-               PoolInfo(stakeToken, 0, 0, pools[i].bonusRatePerSecond, block.timestamp,
pools[i].startTimestamp);

+                           PoolInfo(stakeToken,   0,   0,   pools[i].bonusRatePerSecond,
pools[i].startTimestamp, pools[i].startTimestamp);

                                    stakeTokenToVestingCategoryId[address(stakeToken)]   =
pools[i].vestingCategoryId;

            unchecked {

                ++i;

            }

        }

        currentNumOfPools += len;

    }
```

Status

Resolved

Medium

[M-01] Airdrop#deallocateLockToken - Airdrop Lock Token De-allocation May Break Pending Claims

Description

The `Airdrop::deallocateLockToken` function does not check for excess tokens after distribution. This allows the owner to de-allocate Lock tokens at any time.

Vulnerability Details

Improper validation of `_lockAmount` lets the `owner` `de-allocate` anytime, causing pending user claims to revert due to insufficient Lock tokens.

```
  function deallocateLockToken(uint256 _lockAmount) external onlyOwner nonReentrant {
@>      lockToken.safeTransfer(msg.sender, _lockAmount);
        emit LockTokenDeallocated(msg.sender, _lockAmount);  }
```

Impact

Because there are no restrictions on the function, the `owner` can `de-allocate` whenever he wishes to, which makes users, who are waiting to claim airdrop tokens after tokens get distributed to them, not able to withdraw those distributed Lock tokens.

Recommendation

Only `de-allocate` `Lock` tokens when there are extra tokens, which means only if there are excess tokens after pending rewards are distributed:

```
    function deallocateLockToken(uint256 _lockAmount) external onlyOwner nonReentrant {
      require(totalPendingAirdropAmount  >=  IERC20(lockToken).balanceOf(address(this)) - _lockAmount), "Not enough tokens to deallocate")
        lockToken.safeTransfer(msg.sender, _lockAmount);
        emit LockTokenDeallocated(msg.sender, _lockAmount); }
```

Status

Resolved

[M-02] IndexStaking#initialize - currentNumOfPools Inflation by Duplicate Pools in IndexStaking Contract

Description

Because of no proper validation while adding pools to the `IndexStaking` contract, the owner can add the same pool as many times to increase the `currentNumOfPools` parameter, ending up decreasing the rewards for the staked user across the pools.

Vulnerability Details

The function `IndexStaking::initialize` is used to deploy different pools, so that users can stake assets for rewards. But there is no proper validation on the pool setting in this function. Because the owner can provide the same pool info with a large number of elements in an array. Lack of proper check, whether the pool was already registered or not, the for loop kept setting the same pool, which ended up increasing the `currentNumOfPools`

```
  function initialize(
        address _owner,
        address _validator,
        address _lockonVesting,
        address _lockToken,
        uint256 _currentRewardAmount,
        string memory _domainName,
        string memory _signatureVersion,
        InitPoolInfo[] calldata pools
    ) external initializer {
        ...
```

```
        uint256 len = pools.length;

        for (uint256 i; i < len;) {

            IERC20 stakeToken = pools[i].stakeToken;

              require(address(stakeToken) != address(0), "Index Staking: Zero address not
allowed");

            require(

                  pools[i].bonusRatePerSecond != 0, "Index Staking: Pool bonus rate per
second must be greater than 0"

            );

             require(pools[i].vestingCategoryId != 0, "Index Staking: Vesting category id
must be greater than 0");

            tokenPoolInfo[address(stakeToken)] =

                 PoolInfo(stakeToken, 0, 0, pools[i].bonusRatePerSecond, block.timestamp,
pools[i].startTimestamp);

                                    stakeTokenToVestingCategoryId[address(stakeToken)]   =
pools[i].vestingCategoryId;

            unchecked {

                ++i;

            }

        }

@>      currentNumOfPools += len;

    }
```

Because `currentNumOfPools` is increased, the rewards generated per second decreases, essentially reducing the rewards received by the owner. As you can see in the function `rewardTokenPerSecond`, the current pool returns the number of rewards generated per second is based on the `currentNumOfPools`, so if the owner intentionally increases this value, without that many pools, then the rewards generated are far less than the actual:

```
  function rewardTokenPerSecond(address _stakeToken) public view returns (uint256) {

        PoolInfo storage poolInfo = tokenPoolInfo[_stakeToken];

@>       return (currentRewardAmount * poolInfo.bonusRatePerSecond) / currentNumOfPools /
PRECISION;

    }
```

Impact

The owner can set the same pool with the same parameters by providing a large array of the same elements, increasing the `currentNumOfPools` parameter, which decreases the rewards generated per second. Ended up saving a lot more rewards for the user.

Recommendation

Update in the `IndexStaking::initialize` function,

```
    uint256 len = pools.length;

        for (uint256 i; i < len; ) {

            IERC20 stakeToken = pools[i].stakeToken;

              require(address(stakeToken) != address(0), "Index Staking: Zero address not
allowed");

+             require(tokenPoolInfo[address(stakeToken)].stakeToken == IERC20(address(0)),
"Pool already been created");

            require(

                pools[i].bonusRatePerSecond != 0,

                "Index Staking: Pool bonus rate per second must be greater than 0"

            );

             require(pools[i].vestingCategoryId != 0, "Index Staking: Vesting category id
must be greater than 0");

            tokenPoolInfo[address(stakeToken)] = PoolInfo(

                stakeToken,

                0,
```

```
                0,
                pools[i].bonusRatePerSecond,
                block.timestamp,
                pools[i].startTimestamp
            );
        stakeTokenToVestingCategoryId[address(stakeToken)] = pools[i].vestingCategoryId;
            unchecked {
                ++i;
            }
        }
        currentNumOfPools += len;
```

Status

Resolved

Low

[L-01] Implement a "Recover" Function for Accidental Transfers

Description

Every contract, except `LockToken`, must implement a recover function for accidental transfers. Among them, `IndexStaking` must implement a "recover" function, as it interacts with other tokens to get rewarded for staking the respective stake tokens.

Impact

As there is no recover function within the contracts, if there occurs any accidental transfers, the tokens permanently get locked within the contract. Though it was user mistake for accidental transfer, but upon implementing this function, increases protocol credibility with users

Recommendation

Implement Recover function in `LockonVesting`, `LockStaking`, and mainly in `IndexStaking` contracts.

```
+ function recover(address _token) external onlyOwner {
+            uint256 amount = IERC20(_token).balanceOf(address(this));
+            if (amount > 0) {
+                  IERC20(_token).safeTransfer(msg.sender, amount);
+            }
+ }
```

Status

Acknowledged

[L-02] LockonVesting#initialize - Lack of Proper validation makes Vesting Category override its own Values

Description

Because there is no proper validation on `LockonVesting::initialize`, if any duplicate arrays are provided through input, then the previous vesting category gets overridden with the new vesting category period.

Vulnerability Details

This function checks whether the category value is zero or not, but it does not check whether it has already been assigned or not. Because of this, if there is any duplicate array of different values of the same set, then for the specific category `ID`, the category period gets overridden.

```
   function initialize(
        address _owner,
        address _lockToken,
        uint256[] memory _categoryIds,
        uint256[] memory _vestingPeriods
    ) external initializer {
        require(_owner != address(0), "LOCKON Vesting: owner is the zero address");
            require(_lockToken != address(0), "LOCKON Vesting: lockToken is the zero address");
        require(
            _categoryIds.length == _vestingPeriods.length,
            "LOCKON Vesting: categoryIds and vestingPeriods length mismatch"
        );
        // Initialize Ownable lib and set the owner
        __UUPSUpgradeable_init();
        __Ownable_init(_owner);
```

```
        __ReentrancyGuard_init();

        __Pausable_init();

        lockToken = ILockToken(_lockToken);

        // Set vesting categories

        for (uint256 i = 0; i < _categoryIds.length; ) {

@>              require(_vestingPeriods[i] > 0, "LOCKON Vesting: Vesting period must be
greater than 0");

            vestingCategories[_categoryIds[i]] = _vestingPeriods[i];

            unchecked {

                ++i;

            }

        }

    }
```

Impact

Because of the override of vesting period on the same category `ID`, the period may increase or decrease than intended.

Recommendation

```
  function initialize(

        address _owner,

        address _lockToken,

        uint256[] memory _categoryIds,

        uint256[] memory _vestingPeriods

    ) external initializer {

        ...

        // Set vesting categories

        for (uint256 i = 0; i < _categoryIds.length; ) {
```

```
                require(_vestingPeriods[i] > 0, "LOCKON Vesting: Vesting period must be
greater than 0");
+               require(vestingCategories[_categoryIds[i]] > 0, "LOCKON Vesting already
assigned");
            vestingCategories[_categoryIds[i]] = _vestingPeriods[i];
            unchecked {
                ++i;
            }
        }
    }
```

Status

Acknowledged

[L-03] LockStaking - No Upper Cap on minimumLockDuration, penaltyRate

Description

In the `LockStaking` contract, `minimumLockDuration` is meant to prevent very short deposits, but since it has no upper cap, the owner could set an excessively large value. This would permanently block new users from depositing, limiting the contract's use only to claiming rewards from earlier stakers.

In the `LockStaking` contract, `penaltyRate` is meant to penalize early withdrawals, but since there is no cap, the owner could set it to `100%`, taking all user assets. If set above `100%`, withdrawals would fail entirely, leaving neither the user nor the owner able to recover the locked assets.

Impact

In `LockStaking`, a lack of caps lets the owner set `minimumLockDuration` extremely high, blocking new stakes, and `penaltyRate` up to `100%`, seizing all early withdrawal assets. Values above `100%` could make withdrawals fail entirely.

Recommendation

```
+   uint256 LOCK_DURATION_CAP; //Set value with makers desired
+   uint256 PENALTY_CAP; //Set it to 50%
    function setMinimumLockDuration(uint256 _minimumLockDuration) external onlyOwner {
+      require(_minimumLockDuration < LOCK_DURATION_CAP, "Cannot be greater than the lock duration cap");
        minimumLockDuration = _minimumLockDuration;
                    emit   MinimumLockDurationUpdated(msg.sender,   minimumLockDuration, block.timestamp);}
    function setPenaltyRate(uint256 _penaltyRate) external onlyOwner {
+      require(_penaltyRate < PENALTY_CAP, "Cannot exceed penalty cap");
        penaltyRate = _penaltyRate;
        emit PenaltyRateUpdated(msg.sender, penaltyRate, block.timestamp);
```

Status

Resolved

[L-04] Use Ownable2StepUpgradable Instead of OwnableUpgradable

Description

The contracts within the codebase inherit from `OwnableUpgradeable`, which allows ownership transfers and finalizes in a single step. This means the current owner can immediately assign ownership to any address, including a wrong one (`including address(0)`).

`OpenZeppelin` also provides `Ownable2StepUpgradable`, which introduces a `2-step` ownership transfer mechanism:

1. The present owner calls `transferOwnership(newOwner)` → sets a pending owner.
2. The pending owner must call `acceptOwnership()` to complete the transfer.

Impact

If the owner accidentally transfers ownership, i.e., `address(0)` as it was allowed, to an incorrect or non-functional address, ownership is permanently lost.

Recommendation

Replace inheritance of `OwnableUpgradeable` with `Ownable2StepUpgradable`

Status

Resolved

[L-05] LockToken#_update - Lock Token Should not Receive Its Own Tokens

Description

The function `transfer` / `transferFrom` are used to send `Lock` tokens from one address to another address. So if while airdropping, a malicious `user` / `distributor` set the same user address and ended up receiving most of the airdrop lock tokens, and sent them back to the token contract itself, making them permanently unusable, essentially decreasing the Lock tokens circulating across the chain, as there is no function to burn or mint the tokens.

Impact

If the transferred Lock tokens were set `to` the address as the Lock token address itself, then the tokens get permanently locked within the contract, making it useless it can neither be burned nor be minted back.

Recommendation

```
  function _update(address from, address to, uint256 value) internal virtual override {

        require(!isBlacklisted(msg.sender), "LOCK Token: sender is blocked");

        require(!isBlacklisted(from), "LOCK Token: transfer from is blocked");

        require(!isBlacklisted(to), "LOCK Token: transfer to is blocked");

+       require(to != address(this), "Cannot transfer tokens to token itself");

        super._update(from, to, value);

    }
```

Status

Acknowledged

[L-06] IndexStaking & LockStaking - No Setter Function for currentRewardAmount Variable

Description

The variable `currentRewardAmount` is used to track the number of rewards available for future awarding for staked users. Because there is no setter function for this variable, once all the rewards are distributed, the `currentRewardAmount` will be zero, and any future staking is not possible.

Vulnerability Details

When a user interacts with the `LockStaking` / `IndexStaking` contract, the pool gets updated, which updates the variables that affect the rewards. As shown below in the function `LockStaking::_updatePool`,

```
  function _updatePool() private {
        if (block.timestamp <= startTimestamp) {
            return;
        }
        if (totalLockScore == 0) {
            lastRewardTimestamp = block.timestamp;
            return;
        }
        // Calculate the reward multiplier based on the difference in block timestamps
        uint256 lockReward = getRewardMultiplier(lastRewardTimestamp, block.timestamp);
        if (lockReward == 0) {
            return;
        }
        // Update state data
        rewardPerScore = rewardPerScore + ((lockReward * PRECISION) / totalLockScore);
@>      currentRewardAmount = currentRewardAmount - lockReward;
```

```
        lastRewardTimestamp = block.timestamp;

                emit  PoolDataUpdated(msg.sender,  rewardPerScore,  currentRewardAmount,
lastRewardTimestamp);

    }
```

`currentRewardAmount` get reduced, which means `lockReward` amount of tokens get released, which were distributed among the stakers. So, there is a chance of it reducing to zero, even with updating `bonusRatePerSecond`, so ultimately it reduces to zero, making the contract only be used for claiming rewards.

Impact

Once the `currentRewardAmount` is reduced to zero, then after that point in time, staking Lock tokens isn't possible. And to operate this contract, makers need to redeploy the whole contract.

Recommendation

Implement a setter function when `currentRewardAmount` is zero.

```
function setCurrentRewardAmount(uint256 amount) Public onlyOwner {

      if (currentRewardAmount == 0) {

            currentRewardAmount = amount;

            allocateLockToken(amount);

      }

}
```

Status

Acknowledged

QA

[Q-01] LockStaking - Used the Same Docs as IndexStaking in LockStaking

Description

The comment states that "Allows a user to cancel a staking reward claim order for a specific pool", but in the `LockStaking` contract, there exists only one pool, so there is no meaning in saying it as "specific pool", as it was taken from `IndexStaking`

```
 /**
@>   * @dev Allows a user to cancel a staking reward claim order for a specific pool
     *
     * @param _requestId An ID for the staking reward claim order
     * @param _claimAmount The amount of reward tokens in the claim order
     * @param _signature The signature to validate the cancellation
     */
     function cancelClaimOrder(string calldata _requestId, uint256 _claimAmount, bytes
memory _signature)
        external
        whenNotPaused
    {
       ...
    }
```

Recommendation

Change the function comments in the `LockStaking` contract, as it is the primary way for developer/auditor to refer to the working of the function

Status

Resolved

[Q-02] LockonVesting#deposit - Prevent Depositing Lock Tokens to the Vesting Contract Itself

Description

The function `LockonVesting::deposit` is used to deposit Lock tokens. But because there is no validation on the user address, it can be anything. So, if the depositor is malicious, then he can put the user address as the `LockonVesting` contract address itself, which makes Lock Tokens permanently locked within the contract itself.

Vulnerability Details

As there is no proper check on the user address, if a malicious depositor adds the user address as the contract address itself, then the assigned amount of lock tokens get permanently locked within the contract itself. As there is no recover function, owner can also withdraw those locked lock tokens

```
   function deposit(
@>      address user,
        uint256 amount,
        uint256 categoryId
    ) external onlyDepositGrantedOrOwner whenNotPaused nonReentrant {
        ...]
}
```

Impact

Lack of proper validation on user address creates a void for malicious depositors to permanently lock tokens within the contract itself.

Recommendation

```
    function deposit(
        address user,
        uint256 amount,
```

```
        uint256 categoryId

    ) external onlyDepositGrantedOrOwner whenNotPaused nonReentrant {

        require(amount != 0, "LOCKON Vesting: Vesting amount must be greater than 0");

        require(user != address(0), "LOCKON Vesting: Zero address not allowed");

+       require(user != address(this), "LOCKON Vesting address not allowed");

           require(vestingCategories[categoryId] != 0, "LOCKON Vesting: Category do not
exist");

        require(

            !lockToken.isBlacklisted(user),

            "LOCKON Vesting: User has been banned from all activities in LOCKON Vesting"

        );

        ...

    }
```

Status

Resolved

[Q-03] LockToken - No mint function on LockToken Contract

Description

The LockToken contract has no mint function, preventing new tokens for rewards; an owner-controlled mint would solve this.

Impact

The LockToken contract lacks a mint function, preventing future token creation when needed. Tokens can be transferred without limits, allowing a single address to receive unlimited tokens during airdrops. A malicious receiver could then burn them, reducing totalSupply, despite the protocol's intended 10B cap. Thus, tokens should either be minted as needed or restricted from being burned.

Recommendation

Implement a mint function, where only the owner can be called

```
function mint(address receiver, uint256 amount) external onlyOwner {

      _mint(receiver, amount);

}
```

Make sure users not to burn their tokens

```
   function _update(address from, address to, uint256 value) internal virtual override {

        require(!isBlacklisted(msg.sender), "LOCK Token: sender is blocked");

        require(!isBlacklisted(from), "LOCK Token: transfer from is blocked");

        require(!isBlacklisted(to), "LOCK Token: transfer to is blocked");

+       require(to != address(0), cannot burn the tokens);

        super._update(from, to, value);

    }
```

Status

Acknowledged

[Q-04] Airdrop#distributeAirdropReward - No Per User Cap Limit on User While Distributing Lock Tokens

Description

The function `Airdrop::distributeAirdropReward` is used to distribute Lock tokens to users who have registered their addresses or through front-end interaction.

This function can only be executed by the contract `owner` or `owner-approved` addresses, ensuring that unauthorized or malicious users cannot perform reward distribution. Therefore, the previous concern regarding a "malicious distributor" is not applicable.

Vulnerability Details

As shown in the code, the `distributeAirdropReward` function is permission-protected, allowing only trusted roles (`owner` or approved addresses) to execute it. Because of this access control, external malicious users cannot manipulate reward distribution or redirect `Lock` tokens. No security risk is identified under the current permission model.

```
  function distributeAirdropReward(
        address[] calldata _listUserAddress,
        uint256[] calldata _amounts
    ) external whenNotPaused onlyDistributeGrantedOrOwner {
        uint256 listUserLen = _listUserAddress.length;
        require(listUserLen <= MAX_DISTRIBUTION_COUNT, "Airdrop: Too many addresses");
        require(
            _amounts.length == listUserLen,
            "Airdrop: The list for user address and amount value must have equal length"
        );
        uint256 totalAmount;
        for (uint256 i; i < listUserLen; ) {
                require(_listUserAddress[i] != address(0), "Airdrop: Zero address is not
allowed");
```

```
            require(_amounts[i] != 0, "Airdrop: Zero amount is not allowed");
@>          userPendingReward[_listUserAddress[i]] += _amounts[i];
            totalAmount += _amounts[i];
            unchecked {
                ++i;
            }
        }
        totalPendingAirdropAmount += totalAmount;
                                emit    AirdropRewardDistributed(msg.sender,    totalAmount,
totalPendingAirdropAmount);
    }
```

Impact

Since the function is `access-controlled`, no direct vulnerability exists related to unauthorized distribution or fund drain by external parties. The risk mentioned in the earlier version of this finding is considered `mitigated`.

Recommendation

No further action is required at this point.

However, it is recommended to maintain strict control over the list of approved addresses and periodically review permissions to ensure only trusted operators can access `distributeAirdropReward`.

Status

Acknowledged

Centralisation

The LOCKON project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the LOCKON project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au